

JAVA PROGRAMMING

Complete Exam Notes

**Definitions • Simple Explanation • Key Points •
Syntax • Programs • Output**

UNIT I • UNIT II • UNIT III

Prepared from the topics visible in the supplied syllabus. Unit III is limited to the visible portion of the uploaded image; missing topics are not guessed.

UNIT I — INTRODUCTION TO OOP AND JAVA

1. Overview of OOP

Definition: Object-Oriented Programming (OOP) is a programming approach in which programs are designed using classes and objects.

Simple Explanation: An object contains data and the methods that operate on that data.

Key Points: Class, Object, Encapsulation, Abstraction, Inheritance and Polymorphism.

```
class ClassName {
    dataType variable;

    void methodName() {
        // statements
    }
}
```

Sample Java Program — OOP

```
class Student {
    int mark;

    void display() {
        System.out.println("Mark = " + mark);
    }

    public static void main(String[] args) {
        Student s = new Student();
        s.mark = 90;
        s.display();
    }
}
```

Output:
Mark = 90

2. Object-Oriented Programming Paradigms

OOP organizes a program around **objects and their interactions**.

Main concepts: Classes, Objects, Encapsulation, Abstraction, Inheritance, Polymorphism and Message Passing.

Exam Point: OOP improves code reusability, modularity, security and maintainability.

```
class ClassName {
    dataType variable;

    returnType methodName() {
        // statements
    }
}
```

3. Features of OOP

Encapsulation: Binding data and methods into one unit.

```
class Student {
    private int mark;

    public void setMark(int m) {
        mark = m;
    }

    public int getMark() {
        return mark;
    }
}
```

Abstraction: Hiding implementation details.

```
abstract class Shape {
    abstract void draw();
}
```

Inheritance: Acquiring properties and methods from another class.

```
class Child extends Parent {  
}
```

Polymorphism: One interface/name can represent different forms.

```
Parent obj = new Child();  
obj.show();
```

Exam Point: The four major OOP features are Encapsulation, Abstraction, Inheritance and Polymorphism.

4. Java Buzzwords

Important Java characteristics are **Simple, Object-Oriented, Distributed, Robust, Secure, Architecture Neutral, Portable, High Performance, Multithreaded and Dynamic.**

Exam Point: Java supports Write Once, Run Anywhere (WORA) through bytecode and JVM.

```
Source Code  
↓  
Compiler  
↓  
Bytecode  
↓  
JVM  
↓  
Output
```

5. Overview of Java

Definition: Java is a high-level, object-oriented programming language.

```
class ClassName {  
    public static void main(String[] args) {  
        // statements  
    }  
}
```

Sample Java Program — Basic Java

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello Java");  
    }  
}
```

Output:

Hello Java

6. Data Types, Variables and Arrays

Primitive data types: byte, short, int, long, float, double, char and boolean.

Variable: A named memory location used to store a value.

```
dataType variableName = value;
```

Array: A fixed-size collection of elements of the same type.

```
dataType[] arrayName = new dataType[size];  
  
int[] a = {10, 20, 30};  
  
arrayName[index];
```

Sample Java Program — Array

```
class ArrayDemo {  
    public static void main(String[] args) {  
        int[] a = {10, 20, 30};  
  
        for (int i = 0; i < a.length; i++) {  
            System.out.println(a[i]);  
        }  
    }  
}
```

Output:

10
20

7. Operators

Arithmetic: +, -, *, /, %

Relational: ==, !=, >, <, >=, <=

Logical: &&, ||, !

Assignment: =, +=, -=, *=, /=

Unary: ++, --

Conditional: ?:

Sample Java Program — Operators

```
class OperatorDemo {
    public static void main(String[] args) {
        int a = 10;
        int b = 3;

        System.out.println(a + b);
        System.out.println(a > b);
        System.out.println(a % b);
    }
}
```

Output:

```
13
true
1
```

8. Control Statements

Selection: if, if-else, switch. **Iteration:** for, while, do-while. **Jump:** break, continue, return.

```
if (condition) {
    // statements
}

for (initialization; condition; update) {
    // statements
}

while (condition) {
    // statements
}

do {
    // statements
} while (condition);
```

Sample Java Program — Control Statements

```
class ControlDemo {
    public static void main(String[] args) {
        int n = 5;

        if (n % 2 == 0)
            System.out.println("Even");
        else
            System.out.println("Odd");

        for (int i = 1; i <= 3; i++)
            System.out.println(i);
    }
}
```

Output:

```
Odd
1
2
3
```

9. Programming Structures in Java

The three basic programming structures are **Sequence, Selection and Iteration**.

```
// Sequence
statement1;
statement2;
statement3;

// Selection
if (condition) {
// statements
}

// Iteration
for (initialization; condition; update) {
// statements
}
```

Exam Point: Sequence executes in order; selection chooses a path; iteration repeats statements.

10. Defining Classes in Java

A class is a blueprint for creating objects.

```
class ClassName {
dataType variable;

returnType methodName() {
// statements
}
}

ClassName objectName = new ClassName();
```

Sample Java Program — Class and Object

```
class Student {
int id;
String name;

void display() {
System.out.println(id + " " + name);
}

public static void main(String[] args) {
Student s = new Student();
s.id = 101;
s.name = "Arun";
s.display();
}
}
```

Output:

101 Arun

11. Constructors

A constructor initializes an object. It has the **same name as the class** and has **no return type**.

```
class ClassName {
ClassName() {
// initialization
}

ClassName(dataType value) {
// initialization
}
}
```

Sample Java Program — Parameterized Constructor

```
class Student {
int id;
String name;

Student(int i, String n) {
id = i;
name = n;
}
}
```

```

void display() {
    System.out.println(id + " " + name);
}

public static void main(String[] args) {
    Student s = new Student(101, "Arun");
    s.display();
}
}

```

Output:
101 Arun

12. Methods

A method is a named block of code that performs a specific task.

```

returnType methodName() {
    // statements
}

returnType methodName(dataType parameter) {
    // statements
}

returnType methodName() {
    return value;
}

```

Sample Java Program — Method

```

class MethodDemo {
    static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println(add(10, 20));
    }
}

```

Output:
30

13. Access Specifiers

Access specifiers control the visibility of class members.

```

private int x;
int x; // default
protected int x;
public int x;

```

Exam Point: **private** → same class; **default** → same package; **protected** → package + subclass access; **public** → broadly accessible.

14. Static Members

A static member belongs to the **class** rather than an individual object.

```

static dataType variableName;

static returnType methodName() {
    // statements
}

ClassName.variableName;
ClassName.methodName();

```

Sample Java Program — Static Member

```

class Counter {
    static int count = 0;

    Counter() {

```

```
count++;  
}  
  
public static void main(String[] args) {  
    new Counter();  
    new Counter();  
    System.out.println(count);  
}  
}
```

Output:

2

15. JavaDoc Comments

JavaDoc comments are documentation comments used to describe Java code and generate documentation.

```
/**  
 * Description  
 * @param parameter Description  
 * @return Description  
 */
```

Exam Point: Common tags: @param, @return, @author and @throws.

UNIT II — INHERITANCE, PACKAGES AND INTERFACES

16. Overloading Methods

Method overloading means having the **same method name with different parameter lists**. It is compile-time polymorphism.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Sample Java Program — Method Overloading

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(2, 3));
        System.out.println(c.add(2, 3, 4));
    }
}
```

Output:

5
9

Exam Point: Same name + different parameters = Overloading. Return type alone cannot overload a method.

17. Objects as Parameters

An object can be passed as an argument to a method.

```
returnType methodName(ClassName object) {
    // statements
}

methodName(object);
```

Sample Java Program — Object as Parameter

```
class Student {
    int mark;

    Student(int m) {
        mark = m;
    }

    static void display(Student s) {
        System.out.println("Mark = " + s.mark);
    }

    public static void main(String[] args) {
        Student s = new Student(90);
        display(s);
    }
}
```

Output:

Mark = 90

18. Returning Objects

A method can return an object of a class.

```
ClassName methodName() {  
    return object;  
}  
  
ClassName obj = methodName();
```

Sample Java Program — Returning Object

```
class Number {  
    int value;  
  
    Number(int v) {  
        value = v;  
    }  
  
    static Number create() {  
        return new Number(50);  
    }  
  
    public static void main(String[] args) {  
        Number n = create();  
        System.out.println(n.value);  
    }  
}
```

Output:

50

19. Static, Nested and Inner Classes

A nested class is declared inside another class. A static nested class is declared using static. A non-static nested class is an inner class.

```
class Outer {  
    static class Nested {  
        // members  
    }  
  
    class Inner {  
        // members  
    }  
}  
  
Outer.Nested n = new Outer.Nested();  
  
Outer o = new Outer();  
Outer.Inner i = o.new Inner();
```

Sample Java Program — Nested and Inner Classes

```
class Outer {  
    static class Nested {  
        void show() {  
            System.out.println("Nested Class");  
        }  
    }  
  
    class Inner {  
        void show() {  
            System.out.println("Inner Class");  
        }  
    }  
  
    public static void main(String[] args) {  
        Nested n = new Nested();  
        n.show();  
  
        Outer o = new Outer();  
        Inner i = o.new Inner();  
        i.show();  
    }  
}
```

Output:

Nested Class

Inner Class

20. Inheritance — Basics and Types

Inheritance allows a child class to acquire properties and methods from a parent class.

```
class Parent {  
    // members  
}  
  
class Child extends Parent {  
    // members  
}
```

Types: Single, Multilevel and Hierarchical inheritance. Java does not support multiple inheritance through classes.

Sample Java Program — Inheritance

```
class Animal {  
    void eat() {  
        System.out.println("Eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking");  
    }  
}  
  
public static void main(String[] args) {  
    Dog d = new Dog();  
    d.eat();  
    d.bark();  
}
```

Output:

Eating

Barking

21. super Keyword

The super keyword refers to the **immediate parent class**.

```
super.variableName;  
super.methodName();  
super();
```

Sample Java Program — super Keyword

```
class Parent {  
    int x = 10;  
  
    void show() {  
        System.out.println("Parent Method");  
    }  
}  
  
class Child extends Parent {  
    int x = 20;  
  
    void display() {  
        System.out.println(super.x);  
        super.show();  
    }  
  
    public static void main(String[] args) {  
        Child c = new Child();  
        c.display();  
    }  
}
```

Output:

10

22. Method Overriding

Method overriding occurs when a subclass provides its own implementation of an inherited method with the same signature.

```
class Parent {
    void show() {
        // parent implementation
    }
}

class Child extends Parent {
    @Override
    void show() {
        // child implementation
    }
}
```

Sample Java Program — Method Overriding

```
class Animal {
    void sound() {
        System.out.println("Animal Sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog Sound");
    }
}

public static void main(String[] args) {
    Dog d = new Dog();
    d.sound();
}
}
```

Output:

Dog Sound

23. Dynamic Method Dispatch

Dynamic method dispatch selects an overridden method at **runtime**.

```
Parent reference = new Child();
reference.method();
```

Sample Java Program — Dynamic Method Dispatch

```
class Animal {
    void sound() {
        System.out.println("Animal");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog");
    }
}

public static void main(String[] args) {
    Animal a = new Dog();
    a.sound();
}
}
```

Output:

Dog

Exam Point: Parent reference + child object + overridden method = runtime method selection.

24. Abstract Classes

An abstract class is declared using the abstract keyword. It may contain abstract and concrete methods and cannot be instantiated directly.

```
abstract class Shape {
    abstract void draw();

    void message() {
        // statements
    }
}

class Circle extends Shape {
    void draw() {
        // implementation
    }
}
```

Sample Java Program — Abstract Class

```
abstract class Shape {
    abstract void draw();

    void message() {
        System.out.println("Shape");
    }
}

class Circle extends Shape {
    void draw() {
        System.out.println("Circle");
    }
}

public static void main(String[] args) {
    Circle c = new Circle();
    c.draw();
    c.message();
}
```

Output:

Circle
Shape

25. final with Inheritance

final is used to restrict modification or inheritance.

```
final int x = 10; // cannot reassign

final void show() { // cannot override
}

final class A { // cannot extend
}
```

Exam Point: final variable → cannot reassign; final method → cannot override; final class → cannot extend.

26. Packages and Member Access

A package is a group of related classes and interfaces. Packages organize code, avoid naming conflicts and provide access control.

```
package packageName;

class Student {
}
```

Exam Point: Access modifiers work with packages: private, default, protected and public.

27. Importing Packages

The import statement allows a program to use classes from another package.

```
import packageName.ClassName;

import java.util.*;
import java.util.Scanner;
```

Sample Java Program — Importing a Package

```
import java.util.ArrayList;

class ImportDemo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();

        list.add("Java");
        list.add("OOP");

        System.out.println(list);
    }
}
```

Output:

[Java, OOP]

28. Interfaces

An interface defines a contract that a class implements.

```
interface InterfaceName {
    void methodName();
}

class ClassName implements InterfaceName {
    public void methodName() {
        // implementation
    }
}

class Demo implements Interface1, Interface2 {
}
```

Sample Java Program — Interface

```
interface Printable {
    void print();
}

class Demo implements Printable {
    public void print() {
        System.out.println("Printing");
    }

    public static void main(String[] args) {
        Demo d = new Demo();
        d.print();
    }
}
```

Output:

Printing

UNIT III — EXCEPTION HANDLING AND MULTITHREADING

The uploaded syllabus image is cropped after the beginning of “Java Thread Model”. The notes below therefore cover only the visible Unit III topics and do not guess the missing topics.

29. Exception Handling

An exception is an abnormal event that interrupts the normal flow of program execution.

```
try {
    // risky code
}
catch (ExceptionType e) {
    // handling code
}
finally {
    // cleanup
}
```

Sample Java Program — Exception Handling

```
class ExceptionDemo {
    public static void main(String[] args) {
        try {
            int x = 10 / 0;
            System.out.println(x);
        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        }
    }
}
```

Output:

Cannot divide by zero

Exam Point: try = risky code; catch = handling; finally = cleanup.

30. Multiple Catch Clauses

Multiple catch blocks handle different types of exceptions from one try block.

```
try {
    // statements
}
catch (ExceptionType1 e) {
    // handling
}
catch (ExceptionType2 e) {
    // handling
}
catch (Exception e) {
    // handling
}
```

Sample Java Program — Multiple Catch

```
class MultiCatchDemo {
    public static void main(String[] args) {
        try {
            int[] a = {10, 20};
            System.out.println(a[5]);
        }
        catch (ArithmeticException e) {
            System.out.println("Arithmetic Error");
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Index Error");
        }
        catch (Exception e) {
            System.out.println("Other Error");
        }
    }
}
```

Output:

Array Index Error

31. Nested try Statements

A try block placed inside another try block is called a nested try.

```
try {
    // outer code

    try {
        // inner code
    }
    catch (ExceptionType e) {
        // inner handling
    }

}
catch (ExceptionType e) {
    // outer handling
}
```

Sample Java Program — Nested try

```
class NestedTryDemo {
    public static void main(String[] args) {
        try {
            System.out.println("Outer Try");

            try {
                int x = 10 / 0;
            }
            catch (ArithmeticException e) {
                System.out.println("Inner Catch");
            }

        }
        catch (Exception e) {
            System.out.println("Outer Catch");
        }
    }
}
```

Output:

Outer Try

Inner Catch

32. Java's Built-in Exceptions

Java provides predefined exception classes for common errors.

Examples: `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`, `NumberFormatException` and `ClassCastException`.

```
try {
    // code
}
catch (ExceptionType e) {
    // handling
}
```

Sample Java Program — Built-in Exception

```
class BuiltInDemo {
    public static void main(String[] args) {
        try {
            int n = Integer.parseInt("ABC");
            System.out.println(n);
        }
        catch (NumberFormatException e) {
            System.out.println("Invalid Number");
        }
    }
}
```

Output:

Invalid Number

33. User-defined Exceptions

A user-defined exception is a custom exception created by the programmer, commonly by extending `Exception`.

```
class MyException extends Exception {
    MyException(String message) {
        super(message);
    }
}

throw new MyException("Error");

void check() throws MyException {
    // statements
}

try {
    check();
}
catch (MyException e) {
    System.out.println(e.getMessage());
}
```

Sample Java Program — User-defined Exception

```
class AgeException extends Exception {
    AgeException(String message) {
        super(message);
    }
}

class UserExceptionDemo {
    static void checkAge(int age) throws AgeException {
        if (age < 18)
            throw new AgeException("Age is below 18");

        System.out.println("Eligible");
    }

    public static void main(String[] args) {
        try {
            checkAge(16);
        }
        catch (AgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output:

Age is below 18

34. Multithreaded Programming

Multithreading means executing multiple threads within a program. A thread is a lightweight unit of execution.

Two common approaches: extending `Thread` and implementing `Runnable`.

```
class MyThread extends Thread {
    public void run() {
        // thread task
    }
}

MyThread t = new MyThread();
t.start();
```

Sample Java Program — Thread Class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}
```



```

class ThreadDemo {
public static void main(String[] args) {
MyThread t = new MyThread();
t.start();
}
}

```

Output:

Thread is running

```

class MyTask implements Runnable {
public void run() {
// task
}
}

```

```

MyTask task = new MyTask();
Thread t = new Thread(task);
t.start();

```

Sample Java Program — Runnable

```

class Task implements Runnable {
public void run() {
System.out.println("Task is running");
}
}

class RunnableDemo {
public static void main(String[] args) {
Thread t = new Thread(new Task());
t.start();
}
}

```

Output:

Task is running

35. Java Thread Model

The Java Thread Model provides a standard mechanism for creating and managing threads.

```

Create Thread
↓
start()
↓
Thread begins execution
↓
run()
↓
Task completed

```

Important methods:

```

t.start(); // starts a new thread
t.run(); // contains thread task
Thread.sleep(1000); // pauses current thread
t.join(); // waits for thread to finish

```

Sample Java Program — Java Thread Model

```

class MyThread extends Thread {
public void run() {
System.out.println("Running Thread");
}

public static void main(String[] args) {
MyThread t = new MyThread();
t.start();
}
}

```

Output:

Running Thread

QUICK REVISION — IMPORTANT SYNTAX

Class and Object

```
class A { }

A obj = new A();
```

Constructor and Method

```
A() { }

void show() {
}
```

Overloading and Overriding

```
// Overloading
void add(int a) { }
void add(int a, int b) { }

// Overriding
@Override
void show() { }
```

Inheritance and super

```
class B extends A {
}

super.x;
super.show();
super();
```

Abstract and final

```
abstract class A {
    abstract void show();
}

final int x = 10;
final void show() { }
final class A { }
```

Package and Interface

```
package mypack;

import java.util.*;

interface A {
    void show();
}

class B implements A {
    public void show() { }
}
```

Exception Handling

```
try {
    // risky code
}
catch (Exception e) {
    // handling
}
finally {
    // cleanup
}

throw new MyException("Error");

void check() throws Exception {
}
```

Thread

```
class T extends Thread {  
    public void run() {  
        // task  
    }  
}  
  
T t = new T();  
t.start();
```

MOST IMPORTANT DIFFERENCES

Overloading → same name + different parameters
Overriding → same method + child implementation
Inheritance → child acquires parent members
Abstraction → hides implementation details
Encapsulation → binds data + methods
Polymorphism → one name/interface, different forms
final variable → cannot reassign
final method → cannot override
final class → cannot extend

END OF NOTES

Definitions + Explanation + Key Points + Syntax + Programs + Outputs